Java multithreading

Threads, their lifecycle, how you create and coordinate them (wait/notify, join, interrupt), and the classic traps. The tone is how I would talk each one through with a teammate; the experience questions are fully spoken.

1. What is multithreading, why do we need it, and what does it solve?

So multithreading is **running multiple threads inside one process at the same time**, where a thread is the smallest unit of execution. We need it for two things: **responsiveness** (do slow work in the background so the app does not freeze) and **throughput** (use all the CPU cores and overlap waiting, so while one thread waits on I/O another does real work). The problem it solves is that a single-threaded program does one thing at a time, so it either blocks on slow work or leaves most of the machine idle. Multithreading lets you keep the CPU busy and the app responsive.

2. Process v/s thread, multitasking v/s multithreading, single v/s multi-threaded.

These trip people up because they sound similar:

Comparison	The difference
Process v/s Thread	A process has its own isolated memory and is heavy; a thread lives inside a process and shares its memory, so it is lighter and faster to switch
Multitasking v/s Multithreading	Multitasking runs multiple **processes** at once; multithreading runs multiple **threads** inside one process
Single v/s Multi-threaded	One thread does everything in sequence; multiple threads overlap work, using more cores but needing coordination

The key point on threads: because they **share memory**, they are cheap to communicate but dangerous around shared mutable state, which is where synchronisation comes in.

3. What are the advantages and disadvantages, and when should you avoid multithreading?

Advantages: better CPU use, higher throughput, and a responsive app (background work does not block the UI or the request). **Disadvantages**: it is **hard to get right**, race conditions and deadlocks are subtle and timing-dependent, debugging is painful, and context switching and locking add overhead. When to **avoid** it: when the work is simple and fast (the threading overhead costs more than it saves), when the task is purely sequential, or when the shared state is so tangled that the concurrency bugs would outweigh the gains. My rule: reach for threads when you have real I/O waiting or real parallel CPU work, not by default.

4. How does the JVM support multithreading?

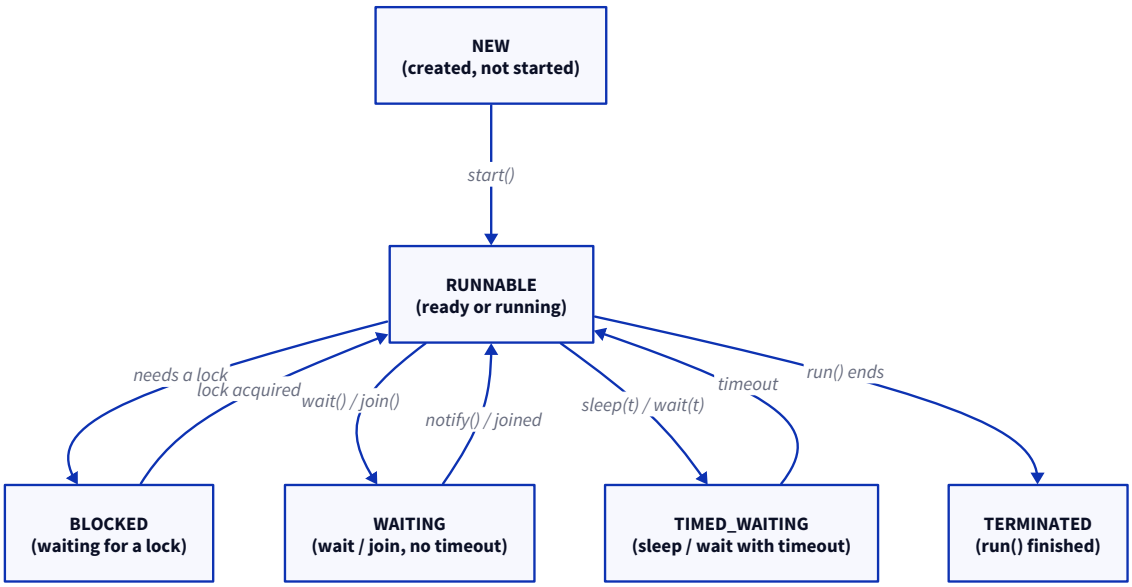
The JVM maps each Java `Thread` onto an **operating-system thread**, so the OS scheduler actually runs them across cores. It gives you the `Thread` class and the `java.util.concurrent` toolkit, and crucially the **Java Memory Model**, which defines the visibility and ordering rules (happens-before) that make `synchronized` and `volatile` mean something across threads. So Java gives you the API and the memory guarantees, and the OS does the real scheduling.

5. Explain the thread lifecycle and its states.

A thread moves through a fixed set of states (the `Thread.State` enum):

- **NEW**: created but `start()` not called yet.
- **RUNNABLE**: eligible to run; this covers both "ready and waiting for the CPU" and "actually running" (Java does not separate a distinct RUNNING state, that is a conceptual sub-state of RUNNABLE).
- **BLOCKED**: waiting to acquire a monitor lock someone else holds.
- **WAITING**: waiting indefinitely for another thread (`wait()` with no timeout, `join()`).
- **TIMED_WAITING**: waiting for a set time (`sleep(t)`, `wait(t)`, `join(t)`).
- **TERMINATED**: `run()` has finished.

On the two distinctions people ask: **NEW v/s RUNNABLE** is "not started yet" v/s "started and eligible to run", and **RUNNABLE v/s RUNNING** is really the same state, RUNNABLE includes running, so the OS decides moment to moment who is on the CPU. You check a thread's state with `thread.getState()`.



6. What are the ways to create a thread, and why prefer `Runnable`?

Two main ways: **extend** `Thread` and override `run()`, or **implement** `Runnable` (or `Callable` if you want a return value) and hand it to a `Thread` or an executor. You **prefer** `Runnable` because Java has single inheritance, so extending `Thread` **burns your one superclass slot** for no reason, whereas `Runnable` leaves you free to extend something else, separates the task from the thread, and drops straight into thread pools and executors. In modern code you rarely create a `Thread` directly at all, you hand a `Runnable` to an `ExecutorService`.

7. start() v/s run(), and why doesn't calling run() create a thread?

This is the classic gotcha. `start()` asks the JVM to create a **new OS thread** and then have that thread call `run()`. `run()` **called directly** is just an ordinary method call, so it runs `run()` **on your current thread**, no new thread at all. That is exactly why calling `run()` does not create a thread: `run()` is just a method, and

`start()` is the only thing that spins up the actual thread. So `myThread.run()` runs synchronously right here; `myThread.start()` runs it concurrently over there.

8. Thread quirks: starting twice, restarting, sharing a Runnable, naming.

Rapid-fire, because these are common yes/no checks:

- **Can a thread be started twice?** No, a second `start()` throws `IllegalThreadStateException`.
- **Can a thread restart after termination?** No, once TERMINATED it is done; you make a new thread.
- **Can multiple threads run the same `Runnable` instance?** Yes, and that is fine, just be careful about the shared state inside it.
- **Can a class extend `Thread` and implement `Runnable`?** Yes (`Thread` already implements `Runnable`), though there is rarely a reason to.
- **Naming**: give threads meaningful names (`Thread.setName` or a naming thread factory), because a good name in a stack trace or a thread dump is worth a lot when you are debugging.

9. How does thread scheduling work, and is it deterministic?

Scheduling is handled by the **OS scheduler**, usually preemptive and priority-aware, so it decides which RUNNABLE thread gets the CPU and for how long. And no, **it is not deterministic**: you cannot predict the exact order threads run, priorities are only hints (and behave differently across platforms), and the order can change run to run. That is why you must **never write code that depends on thread ordering**, if correctness relies on who runs first, you have a bug waiting to happen.

10. What is thread communication, and how do wait(), notify(), and notifyAll() work?

Thread communication is threads **coordinating on shared state** rather than busy-waiting. The classic tools live on `Object`: **`wait()`** makes a thread release the lock and pause until notified; **`notify()`** wakes **one** waiting thread; **`notifyAll()`** wakes **all** of them. You need this so a consumer can sleep until a producer signals "there is work now", instead of spinning in a loop burning CPU. The `notify()` v/s `notifyAll()` call matters: use **`notifyAll()`** when several threads wait on possibly-different conditions, because `notify()` wakes one arbitrary thread and might wake the wrong one, leaving the right one asleep.

11. wait() v/s sleep(), and why the synchronized block, spurious wakeups, and the loop?

Three linked rules that people get wrong. **`wait()` v/s `sleep()`**: `wait()` (on `Object`) **releases the lock** and waits for a notification; `sleep()` (on `Thread`) **keeps the lock** and just pauses for a fixed time. **Why `wait()` must be inside `synchronized`**: you have to **hold the object's monitor** to call `wait()` or `notify()` on it, otherwise you get `IllegalMonitorStateException`, this is what makes the release-and-reacquire safe. **Spurious wakeups**: the JVM is allowed to wake a waiting thread **without any notification**, so you cannot assume "I woke up, therefore the condition is true". **Which is why `wait()` always goes in a `while` loop**, not an `if`: you re-check the condition after waking, and go back to waiting if it is not actually satisfied.

12. Producer-Consumer with wait() and notify().

This is the textbook use of the above. You have a shared bounded buffer, a producer adding items and a consumer taking them. Inside a `synchronized` block: the **producer**, if the buffer is full, calls `wait()` (releasing the lock) until there is space, adds an item, then `notifyAll()`; the **consumer**, if the buffer is empty, `wait()`s until there is something, takes an item, then `notifyAll()`. Both check their condition in a **`while` loop** (not `if`) to handle spurious wakeups and the case where another thread grabbed the item first. In real

code I would just use a `BlockingQueue`, which does all this for you, but the wait/notify version is what interviews want to see.

13. What is join(), how does it work, and how is it different from sleep()?

`join()` makes the current thread **wait until another thread finishes**. So `workerThread.join()` blocks the caller until `workerThread` is done, which is how you wait for background work to complete before moving on. The difference from **`sleep()`**: `join()` waits for a **thread to finish** (however long that takes), while `sleep()` waits a **fixed amount of time** regardless of anything. There is a **timed `join(ms)`** that waits at most that long and then continues even if the thread is still running, and **calling `join()` multiple times** is fine, if the thread already finished, `join()` just returns immediately.

14. What is thread interruption, and how does interrupt() work?

Interruption is Java's **cooperative way to ask a thread to stop**, not to force it. Calling `thread.interrupt()` just **sets the thread's interrupt flag**; it does not kill the thread. If the thread is blocked in something like `sleep()`, `wait()`, or `join()`, that method **throws `InterruptedException`**; otherwise the thread has to check `Thread.interrupted()` itself and decide to stop. The important habit: **do not swallow `InterruptedException`**, if you catch it and do nothing, you have thrown away the stop signal; either handle it (exit the loop) or restore the flag with `Thread.currentThread().interrupt()`.

15. What is a daemon thread, and how is it different from a user thread?

A **daemon thread is a background thread that does not keep the JVM alive**. The rule is: the JVM exits once only daemon threads are left running, so daemons are for background chores (like the garbage collector) that should not stop the program from shutting down. A **user thread** is the opposite, the JVM stays up as long as any user thread is running. You create one by calling **`setDaemon(true)` before `start()`** (you cannot change it after the thread starts). The catch: a daemon can be killed mid-work at shutdown, so do not use one for anything that must finish cleanly.

16. Why is synchronisation needed, and what breaks without it?

Because threads **share memory**, and without coordination two things go wrong. First, **race conditions**: two threads read-modify-write the same data (the classic `count++`) and updates get lost, because that operation is not atomic. Second, **visibility**: without synchronisation (or `volatile`), one thread may not see another's write at all, because of caching and reordering under the memory model. `synchronized` fixes both, it gives **mutual exclusion** (one thread in the critical section at a time) and **visibility** (a happens-before guarantee so changes are seen). So without it you get corrupted data and stale reads that only show up under load.

17. Deadlock v/s livelock.

Both mean "no progress", but for different reasons. **Deadlock** is when threads are **stuck waiting on each other's locks** forever, thread A holds lock 1 and wants lock 2, thread B holds lock 2 and wants lock 1, so neither moves. **Livelock** is when threads are **not blocked but still make no progress**, they keep reacting to each other and retrying (like two people stepping side to side in a corridor), so they are busy but going nowhere. The tell: a deadlocked thread is asleep (BLOCKED), a livelocked thread is burning CPU. You avoid deadlock mainly by **acquiring locks in a consistent order**.

18. What is ThreadLocal, how does it work, and what is the cleanup trap?

`ThreadLocal` gives **each thread its own private copy of a variable**, so there is no sharing and no need to synchronise it. It was introduced for **per-thread context**: things like a per-request user or transaction id, or a `SimpleDateFormat` instance (which is not thread-safe) that each thread can own its own copy of. **Internally**, the value is stored in a `ThreadLocalMap` that hangs off the `Thread` object itself, keyed by the ThreadLocal, so it lives as long as the thread does. And that is exactly the **cleanup trap**: in a **thread pool**, threads are reused forever, so a value you set stays attached to the pooled thread and both **leaks memory and leaks stale data into the next task**. So you must call **`remove()`** in a `finally` block when you are done. *(This is the ThreadLocal mistake that bites people in production.)*

19. Describe a production multithreading issue, and have you debugged deadlocks or contention?

The pattern I have hit is a **lost update on shared state**: two threads incrementing or updating the same object without proper synchronisation, so counts came out wrong intermittently, the kind of bug that vanishes the moment you attach a debugger because the timing changes. The way I worked it was a **thread dump** (which shows exactly what each thread is doing and catches a deadlock as two threads each BLOCKED on a lock the other holds), plus narrowing down the shared state, and the fix was moving to the right concurrent tool, a `ConcurrentHashMap` or an `AtomicLong` instead of a plain field, or fixing the lock ordering. For **contention**, the tell was threads piling up BLOCKED on one lock, and the fix was shrinking the synchronised section or using a lock-free structure. The lesson I keep: reach for `java.util.concurrent` before hand-rolling locks. *(Adapt to a real incident you owned.)*

20. What multithreading best practices does your team follow?

The ones I would call out: **prefer `java.util.concurrent`** (executors, `ConcurrentHashMap`, `BlockingQueue`, atomics) over raw threads and manual `wait` / `notify`; **never `new Thread()` per task**, use a pool; **keep synchronised sections small** and never do I/O while holding a lock; **acquire locks in a consistent order** to avoid deadlock; **always clean up `ThreadLocal`** with `remove()`; **do not swallow `InterruptedException`**; and **never depend on thread scheduling order**. Most of the theme is "use the high-level tools and keep the locked region tiny." *(Adapt to your own team.)*

21. Common mistakes.

- **Calling `run()` instead of `start()`**, which runs on the current thread with no new thread created.
- **Starting the same thread twice** (`IllegalThreadStateException`).
- **Ignoring `InterruptedException`** by swallowing it, which loses the cancellation signal.
- **Calling `wait()` outside a `synchronized` block** (`IllegalMonitorStateException`).
- **Using `notify()` instead of `notifyAll()`** when several threads wait on different conditions, so the wrong one wakes.
- **Synchronising an unnecessarily large block**, which kills concurrency.
- **Holding a lock while doing I/O**, which stalls every other thread waiting on it.
- **Creating too many threads** instead of using a bounded pool.
- **Forgetting to clean up `ThreadLocal`**, which leaks memory and stale data in thread pools.
- **Assuming thread scheduling order is predictable**, which it never is.